

# GPU Thermal Anomaly Detection

## Statistical Hypothesis Testing versus Fixed-Threshold Alerting on Simulated GPU Telemetry

Amit

September 2026

Code: R (base and `optim`)

### Abstract

This report presents and evaluates a simple anomaly-detection pipeline for GPU thermal telemetry. A synthetic dataset of 10,000 temperature readings was constructed, comprising 9,850 observations drawn from a Gaussian model of normal operating temperature and 150 injected readings forming five distinct thermal excursion episodes. Two alerting strategies were evaluated against this known ground truth: a fixed-threshold rule (temperature exceeding 90°C) and a statistical alert derived from maximum likelihood estimation (MLE) of the normal-operating distribution, followed by a one-sided hypothesis test applied to each new reading. The statistical alert achieved a 92.0% detection rate at a 1.08% false alarm rate, compared with a 10.7% detection rate at a 0% false alarm rate for the fixed threshold. These results indicate that explicitly modeling the normal-operating distribution substantially improves the detection of anomalous behavior relative to a single absolute cutoff, at a modest and tunable increase in false alarm rate.

## 1 Problem Statement

GPUs in data-center and ML-training environments constantly emit thermal telemetry, and a common way to watch for overheating is a fixed threshold rule: raise an alert whenever a reading exceeds some absolute cutoff. It's simple and easy to explain, but it treats every reading below the cutoff as equally "fine." It can't tell a reading that's merely a little warm apart from one that's far outside the system's normal operating range, as long as both stay under the line. This project asks whether a statistically grounded alert, one that first learns what "normal" looks like from reference data and then flags new readings that are improbable under that model, can detect more of a thermal anomaly episode than a fixed cutoff can. I use a controlled synthetic benchmark where the true anomalies are known exactly, so detection performance can be measured precisely rather than estimated.

## 2 Data and Simulation Design

The dataset is fully synthetic and generated by `simulate_data.R` with a fixed random seed (`set.seed(3)`) for exact reproducibility.

The normal data (9,850 points) are i.i.d. draws from  $\text{Normal}(\mu = 67^\circ\text{C}, \sigma = 1^\circ\text{C})$ , representing baseline GPU temperature under normal load. These are saved separately as `normal_data.csv` and used later purely as a reference sample for parameter estimation, a stand-in for a historical "known normal" calibration window.

On top of that baseline sit five anomaly episodes (150 points total, 30 points each). Each one is a ramp up and back down, two 15-point linear segments, peaking somewhere between roughly 90°C and 93.5°C, spliced into the normal sequence at five fixed positions. The combined telemetry (10,000 points) is saved as `telemetry_data.csv`, and this is the file both detectors actually run on. By design, anomalies make up 1.5% of the data (150 of 10,000).

The exact index range of every episode is kept for evaluation only; it is never used to fit either detector, which avoids any leakage between training and evaluation.

Table 1: True anomaly episode locations and approximate peak temperatures.

| Episode | Index range | Peak temp (approx.) |
|---------|-------------|---------------------|
| 1       | 2,200–2,229 | 91.5°C              |
| 2       | 3,130–3,159 | 93.5°C              |
| 3       | 4,960–4,989 | 90.5°C              |
| 4       | 7,490–7,519 | 92.0°C              |
| 5       | 8,320–8,349 | 93.5°C              |

### 3 Methodology

#### 3.1 Fixed-threshold baseline

A reading is flagged whenever it exceeds an absolute cutoff:

$$\text{alert}(i) = \begin{cases} 1 & \text{if } x_i > 90^\circ\text{C} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

This is the naive rule the statistical approach is benchmarked against.

#### 3.2 Statistical alert: MLE plus a one-sided hypothesis test

The statistical alert works in three stages.

**Fit the normal-operation distribution.** Using the reference sample of known-normal readings, I estimate  $\mu$  and  $\sigma$  for a  $\text{Normal}(\mu, \sigma)$  model by maximum likelihood, implemented by numerically minimizing the negative log-likelihood with R’s `optim()` rather than relying on the closed-form sample mean and standard deviation. This gives fitted values  $\hat{\mu} \approx 66.995^\circ\text{C}$  and  $\hat{\sigma} \approx 0.998^\circ\text{C}$ , essentially identical to the closed-form sample estimates (mean = 66.994, sd = 1.003), which confirms the optimizer converged correctly.

**Score each telemetry reading.** For every observation  $x_i$ , I compute a z-score relative to the fitted distribution and its one-sided upper-tail probability, the chance a genuinely normal reading would be this high or higher:

$$z_i = \frac{x_i - \hat{\mu}}{\hat{\sigma}}, \quad p_i = P(Z > z_i) = 1 - \Phi(z_i) \quad (2)$$

**Flag by significance.** An alert is raised whenever  $p_i < \alpha = 0.01$ , i.e. the reading is significant at the 1% level: fewer than 1 in 100 truly normal observations would be this extreme by chance. This is effectively a one-sided z-test applied independently to every reading, using a reference distribution learned once from normal-operation data rather than a hand-picked absolute cutoff.

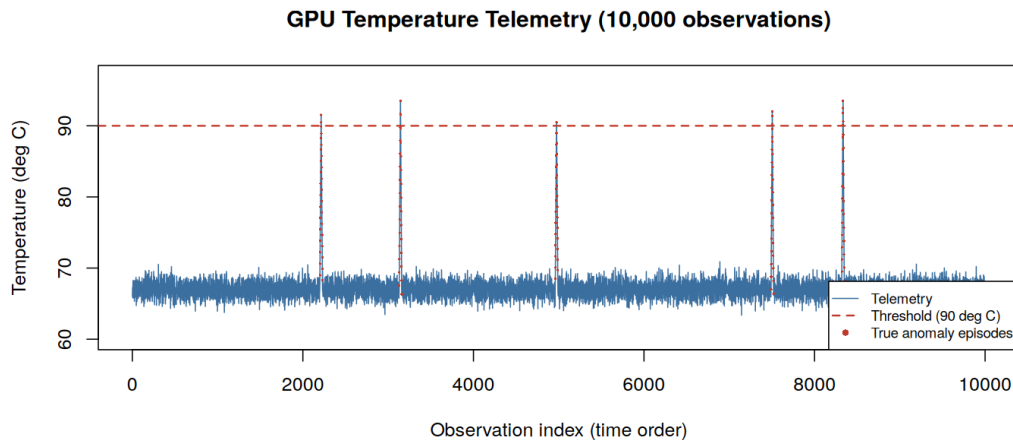


Figure 1: Simulated telemetry (10,000 points). Red dots mark the five true anomaly episodes (150 points); the dashed line is the 90°C fixed threshold. Most episode points sit below the threshold even though they are far outside the normal operating band.

## 4 Evaluation Framework

Each detector’s output is compared point by point against the ground-truth episode labels using a standard confusion matrix: true positives (anomaly, correctly flagged), false negatives (anomaly, missed), true negatives (normal, correctly ignored), and false positives (normal, incorrectly flagged). Two summary metrics are reported:

$$\text{Detection Rate} = \frac{TP}{TP + FN}, \quad \text{False Alarm Rate} = \frac{FP}{FP + TN} \quad (3)$$

Detection rate is the share of true anomalies caught (recall); false alarm rate is the share of normal readings incorrectly flagged.

## 5 Results

Table 2: Confusion-matrix results and summary metrics for both detectors.

| Metric                  | Statistical Alert | Threshold Alert |
|-------------------------|-------------------|-----------------|
| True Positives (TP)     | 138               | 16              |
| False Negatives (FN)    | 12                | 134             |
| True Negatives (TN)     | 9,744             | 9,850           |
| False Positives (FP)    | 106               | 0               |
| <b>Detection Rate</b>   | <b>92.0%</b>      | <b>10.7%</b>    |
| <b>False Alarm Rate</b> | <b>1.08%</b>      | <b>0.0%</b>     |

Taken together, the statistical rule flags 2.44% of all readings (244 of 10,000), close to the true 1.5% anomaly rate, while the threshold rule flags only 0.16% of readings (16 of 10,000).

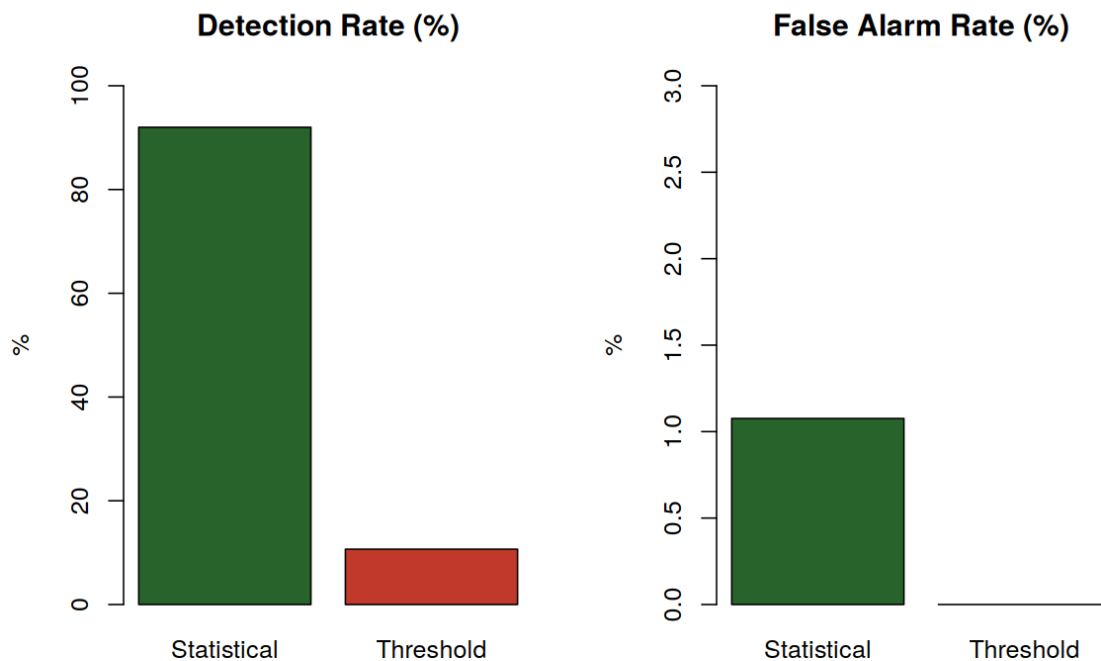


Figure 2: Detection rate and false alarm rate, statistical versus fixed-threshold alert.

## 6 Discussion

The gap in detection rate comes down to the shape of the injected anomalies. Of the 150 true anomaly readings, only 16 (10.7%) ever cross 90°C, the very peak of each ramp. The other 134 readings that make up each episode’s rise and fall sit, in absolute terms, under the fixed cutoff, even though they are many standard deviations above the roughly 67°C baseline and are therefore extremely improbable under normal operation. A fixed threshold has no way to use that information: it only asks whether a reading crosses one hand-picked line. A distribution-aware test asks a different, more informative question, how surprising is this reading given what normal actually looks like, and that is what lets it recover most of each episode rather than just its tip.

That same sensitivity is also the source of its cost. Because a Gaussian tail test will, by construction, occasionally flag genuinely normal noise that happens to land in the upper tail, the statistical alert carries a non-zero false alarm rate (1.08%) where the threshold has none. In an operational setting this trade-off is tunable: lowering  $\alpha$  reduces false alarms at some cost to detection rate, and raising it does the opposite. The single operating point reported here ( $\alpha = 0.01$ ) is a design choice, not a fixed law.

## 7 Limitations and Future Work

- **No temporal structure.** Every reading is tested independently against the reference distribution, so the method has no memory and cannot use the shape of a ramp, a run of several consecutive mild deviations, as evidence in itself. Windowed z-scores, EWMA control charts, or

CUSUM-style accumulation of mild deviations are natural extensions.

- **Single operating point.** Results are reported at one significance level ( $\alpha = 0.01$ ). A full ROC-style scan across  $\alpha$  would characterize the detection-rate and false-alarm-rate trade-off curve rather than a single point on it.
- **Fixed threshold as baseline only.** The 90°C cutoff is a naive comparison point representative of a simple monitoring rule, not a fully tuned baseline; sweeping the cutoff would sharpen the comparison.
- **Episode-level detection latency is not measured.** Ground truth is defined per observation, so a natural follow-on metric is how many observations into an episode the statistical alert first fires.
- **Synthetic data.** Real GPU telemetry likely has richer structure, load-dependent variance, workload change points, sensor drift, that this Gaussian, i.i.d. benchmark does not capture. Validating on real hardware logs, and comparing against unsupervised methods such as isolation forests or autoencoder reconstruction error, is a natural next step.

## 8 Conclusion

On this synthetic benchmark, explicitly modeling what “normal” GPU temperature looks like, via MLE, and testing new readings against that model catches the large majority of injected anomalies (92%) at a small, controllable false-alarm cost (1.08%), compared to a fixed threshold that only catches the extreme tip of each anomaly episode (10.7%) but never raises a false alarm. The exercise is a compact, reproducible demonstration of how classical statistical inference, maximum likelihood estimation paired with a one-sided hypothesis test, applies directly to a practical telemetry-monitoring problem, and of where its assumptions (independence, a static Gaussian normal-operating model) leave room for improvement.

## 9 Reproducing This Analysis

All code is written in base R. `simulate_data.R` generates `telemetry_data.csv` and `normal_data.csv` under a fixed seed (`set.seed(3)`); `analysis.R` performs the MLE fit, computes both alerts, and reproduces every figure and number in this report exactly.